

Java Full Stack Developer

Complete Roadmap — Start to End

Targets the current generation of tools as of late 2026: Java 25 (LTS) and Spring Framework 7 / Spring Boot 4. Phases already covered are marked, with personalized notes on what moves faster given existing experience with React/Next.js, Redis, Docker, and AWS/boto3.

Time Estimate Summary

Phase group	Est. new hours	Note
Advanced Java & SQL (2–3)	~30 hrs	Virtual threads + streams are the bulk
JDBC / Servlets / JSP (4)	~10 hrs	Mostly covered via college coursework
Spring Core, Boot, MVC/REST (7–9)	~48 hrs	Moves fast once IoC clicks
Data Access: JPA/Hibernate (10)	~22 hrs	Often underestimated — N+1, lazy loading take practice
Security / JWT / OAuth2 (11)	~18 hrs	
Testing (12)	~15 hrs	
Caching & Async (13)	~9 hrs	Faster — Redis already known
Microservices & Messaging (14–15)	~34 hrs	Steepest new-concept phases
DevOps & Deployment (16)	~18 hrs	AWS CLI/boto3 already set up — cloud piece is faster
Capstone (17)	~30 hrs	Can fold into existing RateGate project

Total: ~234 new hours. At 10–12 hrs/week alongside DSA and coursework: roughly 5–7 months. At 30 min/day with no gaps: roughly 15–16 months — a mixed rhythm (20–30 min weekdays + one weekend project block) is more realistic and effective than flat daily minutes.

Detailed Phase-by-Phase Syllabus

Phase 0 — Programming Fundamentals	TO LEARN
— What is programming; compiler vs interpreter; JVM, JRE, JDK — Variables and data types (primitive vs reference types) — Operators (arithmetic, relational, logical, bitwise) — Control flow: if-else, switch, for/while/do-while loops — Arrays (1D and 2D) — Methods: declaration, parameters, return types, overloading	
Phase 1 — Core Java / OOP	ALREADY DONE

- Classes and objects, constructors, this keyword
- Inheritance, super keyword, method overriding
- Polymorphism (compile-time vs runtime)
- Abstraction: abstract classes vs interfaces
- Encapsulation, access modifiers
- Static vs instance members, static blocks
- Packages and access control
- Wrapper classes, autoboxing/unboxing
- String, StringBuilder, StringBuffer — and why immutability matters
- Exception handling: try-catch-finally, checked vs unchecked, custom exceptions, try-with-resources
- Collections framework: List, Set, Map, Queue/Deque, Iterator, Comparable vs Comparator
- Generics: generic classes, methods, bounded types

Phase 2 — Advanced Java

TO LEARN

- Java 8+ features: lambdas, functional interfaces, Streams API, Optional, method references
- Multithreading: Thread, Runnable, lifecycle, synchronization, wait/notify
- Concurrency utilities: ExecutorService, Callable/Future, java.util.concurrent basics
- Virtual Threads (Project Loom) — now core, not optional in Java 25
- Structured concurrency, pattern matching for switch / record patterns
- File I/O: File class, byte vs character streams, BufferedReader/Writer
- JVM memory basics: stack vs heap, garbage collection overview

Virtual threads mean most new Spring projects skip reactive/WebFlux entirely unless doing real-time streaming.

Phase 3 — Database & SQL

TO LEARN

- RDBMS concepts, ER diagrams, primary/foreign keys
- SQL basics: DDL, DML, DCL, TCL
- SELECT, WHERE, ORDER BY, GROUP BY, HAVING
- Joins: inner, left, right, full, self-join
- Subqueries, aggregate functions
- Indexes and why they matter for performance
- Normalization (1NF–3NF)
- Transactions and ACID properties
- Hands-on with MySQL or PostgreSQL and a GUI client

Phase 4 — JDBC & Server-Side Java Foundations

TO LEARN

- JDBC architecture: Driver, Connection, Statement, PreparedStatement, ResultSet
- Connection pooling concepts (HikariCP)
- Writing CRUD operations with raw JDBC
- Servlets: lifecycle, HttpServlet, doGet/doPost, ServletConfig/Context
- Session management: cookies, HttpSession
- JSP: lifecycle, JSTL, expression language (conceptual — don't over-invest)
- MVC pattern via Servlet + JSP (what Spring MVC later abstracts)

Phase 5 — Build Tools & Version Control

ALREADY DONE

- Maven: POM structure, dependency management, build lifecycle, plugins
- Git: init, clone, add, commit, push, pull
- Branching strategies, merging, resolving conflicts
- GitHub workflow: pull requests, issues, writing a real README

Phase 6 — Frontend Fundamentals

ALREADY DONE

- HTML structure, semantic tags
- CSS: box model, flexbox, grid, responsive design
- JavaScript: DOM manipulation, events, async/await, fetch/AJAX
- React/Next.js: components, props, state, hooks, routing, REST API calls, Context API

Phase 7 — Spring Framework Core

TO LEARN

- IoC and Dependency Injection concepts
- ApplicationContext vs BeanFactory
- @Component, @Service, @Repository, @Controller
- @Autowired: constructor vs setter vs field injection
- Bean scopes: singleton, prototype
- Bean lifecycle: @PostConstruct, @PreDestroy
- XML-based vs Java-based vs annotation-based configuration

Phase 8 — Spring Boot

TO LEARN

- Target Spring Boot 4 / Spring Framework 7 — current generation as of late 2026
- @SpringBootApplication and auto-configuration
- application.properties / yml, profiles (dev/prod)
- Spring Boot starters
- CommandLineRunner / ApplicationRunner
- Spring Boot Actuator: health checks, metrics
- Scaffolding with Spring Initializr
- GraalVM native image basics — a genuine 2026 differentiator skill

Phase 9 — Spring MVC & REST APIs

TO LEARN

- DispatcherServlet request flow
- @RestController, @GetMapping/@PostMapping etc.
- @RequestBody, @PathVariable, @RequestParam
- DTOs and separating API shape from entity shape
- Validation: @Valid, Bean Validation annotations
- Centralized exception handling: @ControllerAdvice, @ExceptionHandler
- Filters vs Interceptors
- CORS configuration
- Pagination, sorting, basic API versioning

Phase 10 — Data Access with Spring

TO LEARN

- Spring JDBC / JdbcTemplate
- Hibernate fundamentals: entity mapping, persistence context, L1 cache
- JPA annotations: @Entity, @Id, @GeneratedValue, @Column
- Relationships: @OneToOne, @OneToMany, @ManyToOne, @ManyToMany
- Lazy vs eager loading, the N+1 problem, @EntityGraph
- Spring Data JPA: JpaRepository, derived queries, JPQL, @Query
- Transactions: @Transactional, propagation, isolation levels

Usually the phase people underestimate — budget extra time for debugging relationships.

Phase 11 — Spring Security

TO LEARN

- Authentication vs authorization
- SecurityFilterChain, UserDetailsService
- Password encoding (BCrypt)
- Stateless auth with JWT: structure, generation, validation, refresh tokens
- Role-based access control, method-level security
- OAuth2 / OIDC basics, social login, PKCE

Phase 12 — Testing

TO LEARN

- JUnit 5: annotations, assertions, lifecycle
- Mockito: mocking dependencies, when/verify
- MockMvc for controller-layer testing
- @SpringBootTest, @WebMvcTest, @DataJpaTest
- Testcontainers: integration testing against a real, disposable database

Phase 13 — Caching & Async

TO LEARN

- Spring Cache abstraction: @Cacheable, @CacheEvict
- Redis integration with Spring
- Async processing: @Async, CompletableFuture

Faster for you — you already know Redis manually from your rate limiter project.

Phase 14 — Microservices

TO LEARN

- Monolith vs microservices trade-offs
- Service discovery: Eureka
- API Gateway: Spring Cloud Gateway
- Inter-service communication: RestTemplate, WebClient, Feign
- Centralized configuration: Spring Cloud Config
- Resilience basics: circuit breakers (Resilience4j)

Phase 15 — Messaging

TO LEARN

- Message queue concepts, sync vs async communication
- Kafka or RabbitMQ: producer/consumer fundamentals
- Event-driven architecture basics

Phase 16 — DevOps & Deployment

TO LEARN

- Docker: images, containers, Dockerfile
- docker-compose: running app + MySQL + Redis together
- CI/CD: GitHub Actions — tests and builds on every push
- Cloud deployment basics: deploying a Spring Boot app to AWS or GCP
- Environment variables and secrets management

AWS CLI and boto3/S3 scripting already set up — the cloud piece moves faster; Docker and GitHub Actions are the genuinely new parts.

Phase 17 — Capstone Project

TO LEARN

- Full-stack app: React/Next.js frontend + Spring Boot backend + database + auth + Docker + cloud deployment
- Portfolio polish: clean commit history, real README, architecture notes

Can fold directly into the existing RateGate project instead of building separately.

Phase 18 — AI Integration for Backend Developers

TO LEARN

- Calling LLM APIs (OpenAI, Claude, etc.) from a Spring backend, handling structured responses
- Spring AI fundamentals: mapping LLM responses directly to Java records/objects
- LangChain4j basics: orchestrating LLM pipelines, the AiServices pattern
- Prompt engineering fundamentals for reliable, structured outputs
- Embeddings, vector search, and basic RAG (retrieval-augmented generation) in Java
- Building simple AI agents: routing, decision-making, guardrails, observability
- MCP (Model Context Protocol) server basics using Spring AI's MCP starters — exposing an existing REST service as an AI-callable tool
- Designing endpoints that expose AI capabilities safely (rate limiting, cost control, input validation)

Treat this as a natural extension of Phase 9 (REST APIs) rather than a separate track — an MCP server is largely a well-described REST service. Do this only after Phases 1–17 are solid: fundamentals beat AI-dabbling for fresher hiring, and this layer moves fast enough that framework specifics will keep shifting — pin versions and check docs when you get here rather than treating any one tutorial as permanent truth.

Where things stand: Phases 1, 5, and 6 are done. Phases 0, 2 (partially), and 3 are baseline literacy to shore up. Phases 7–17 are the core path — same as the roadmap tracker, laid out here in full topic detail. Phase 18 (AI integration) is a 2026 market addition, best approached after Phase 9 and once the core path is solid — not a substitute for it.